Dynamic Array - 2

Data Structures & Object-Oriented Programming



Table of Contents

- Amortized Analysis
- 2D Arrays
- Search เป็น O(n)
- Insert/Delete กลาง array
- stack

ทบทวน: DynamicArray

- โครงสร้างภายใน มี 3 attribute ที่ทำงานร่วมกัน คือ
  - _capacity (พื้นที่ที่จองไว้), _size (ข้อมูลจริงที่มี), และ _data (พื้นที่เก็บข้อมูลดิบ
- ใช้งานได้เหมือน Python list ผ่าน dunder methods ที่เขียนไว้ครบแล้ว
  - len(arr), arr[i], arr[i] = value, print(arr), repr(arr)
- เพิ่ม-ลบข้อมูลได้ทุกตำแหน่ง
  - **append(value)** — เพิ่มท้าย พร้อม _resize() อัตโนมัติเมื่อเต็ม (doubling strategy)
  - **insert_at(index, value)** — แทรกตำแหน่งใดก็ได้ (เลื่อนข้อมูลขวาไปซ้าย)
  - **remove_at(index) / pop()** — ลบตำแหน่งใดก็ได้ / ลบตัวสุดท้าย

```
arr = DynamicArray()
arr.append(10)
arr.append(20)
arr.insert_at(1, 99)
print(arr)          # [10, 99, 20]
print(len(arr))     # 3
```

- คำถามที่ยังไม่ได้ตอบ
- append() บางครั้งเร็วมาก (ใส่ค่าเข้าไปในช่องว่าง) แต่บางครั้งช้า (ต้อง resize คัดลอกข้อมูลทั้งหมด)
  - ทำไมยังพูดได้ว่า append เป็น O(1)?

Amortized Analysis
(การวิเคราะห์ต้นทุนเฉลี่ย)

ข้อขัดแย้งที่ดูเหมือนจะเกิดขึ้น

- append บางครั้งเร็วมาก (แค่ใส่ค่าเข้าไปในช่องที่ว่างอยู่) แต่บางครั้งกลับช้ามาก (ต้อง resize ซึ่งต้องคัดลอกข้อมูลทั้งหมด)
- คำถามที่ต้องตอบให้ได้คือ
  - ถ้า resize มีต้นทุนสูงถึง O(n) แล้วทำไมยังพูดว่า “append เป็น O(1)” ได้?
- คำตอบ คือ เพราะ O(1) ไม่ได้หมายถึง “ทุกครั้งที่ append ใช้เวลาคงที่” แต่หมายถึง “เมื่อเฉลี่ยต้นทุนของการ append หลายๆ ครั้งรวมกัน จะได้ค่าเฉลี่ยต่อครั้งเป็น O(1)”
  - วิธีคิดแบบนี้เรียกว่า Amortized Analysis (การวิเคราะห์ต้นทุนถัวเฉลี่ย)
  - คำว่า “amortized” มาจากคำว่า amortize ที่แปลว่า “กระจายต้นทุนออกไป”
    - คล้ายกับการผ่อนชำระหนี้ก้อนใหญ่ออกเป็นงวดย่อยๆ แทนที่จะจ่ายทีเดียวทั้งหมด

วิธีคิดต้นทุนรวมของ n Appends

- สมมติเริ่มต้นจาก capacity = 1 แล้ว append ทั้งหมด n ครั้ง
- ต้นทุนจากการ resize ทั้งหมด คือ ผลรวมของอนุกรมเรขาคณิต
  - ต้นทุน resize = $1 + 2 + 4 + 8 + \ldots \leqslant 2n$
- ต้นทุนจากการ append ปกติ (ที่ไม่ต้อง resize) แต่ละครั้งใช้เวลาคงที่ (แค่ใส่ค่าลงช่องว่าง) ดังนั้นถ้า append ทั้งหมด n ครั้ง ต้นทุนส่วนนี้คือ
  - ต้นทุน append ปกติ = n
- ต้นทุนรวมทั้งหมด เมื่อบวกสองส่วนเข้าด้วยกัน
  - ต้นทุนรวม = $n + 2n = 3n = O(n)$
- ต้นทุนเฉลี่ยต่อการ append หนึ่งครั้ง คำนวณโดยเอาต้นทุนรวมหารด้วยจำนวนครั้งที่ทำ
  - ต้นทุนเฉลี่ยต่อครั้ง = $O(n) / n = O(1)$
- นี่คือที่มาของคำว่า “O(1) amortized” — แม้บางครั้งจะมีต้นทุนสูง (ตอน resize) แต่เมื่อเฉลี่ยกับครั้งอื่นๆ ที่ถูกกว่ามาก (ตอนไม่ resize) แล้ว ค่าเฉลี่ยต่อครั้งกลับกลายเป็นค่าคงที่

ลองคำนวณด้วยตัวเลขจริง n=8

ลองไล่คำนวณต้นทุนจริงของการ append 8 ครั้งติดกัน โดยเริ่มจาก capacity = 1

Append ครั้งที่	เกิด resize?	Capacity ก่อน→หลัง	ต้นทุนการ copy (ถ้า resize)	ต้นทุนการเพิ่มค่าใหม่	ต้นทุนรวมครั้งนี้
1	ไม่	1→1	0	1	1
2	ใช่	1→2	1	1	2
3	ใช่	2→4	2	1	3
4	ไม่	4→4	0	1	1
5	ใช่	4→8	4	1	5
6	ไม่	8→8	0	1	1
7	ไม่	8→8	0	1	1
8	ไม่	8→8	0	1	1

- ต้นทุนรวมทั้งหมด = 1 + 2 + 3 + 1 + 5 + 1 + 1 + 1 = 15
- ต้นทุนเฉลี่ยต่อการ append หนึ่งครั้ง = 15 / 8 = 1.875
  - ใกล้เคียงกับค่าคงที่มาก ไม่ได้โตขึ้นตามจำนวนครั้งที่ append
- หากลองขยายไปจนถึง n ที่มีขนาดใหญ่มากๆ (เช่น append ล้านครั้ง) ค่าเฉลี่ยนี้จะยิ่งเข้าใกล้ค่าคงที่ตัวหนึ่งมากขึ้นเรื่อยๆ (ในทางทฤษฎีจะเข้าใกล้ 3) นี่คือหลักฐานที่จับต้องได้ว่าทำไมจึงเรียกว่า O(1) amortized

Amortized ไม่ใช่ Average Case

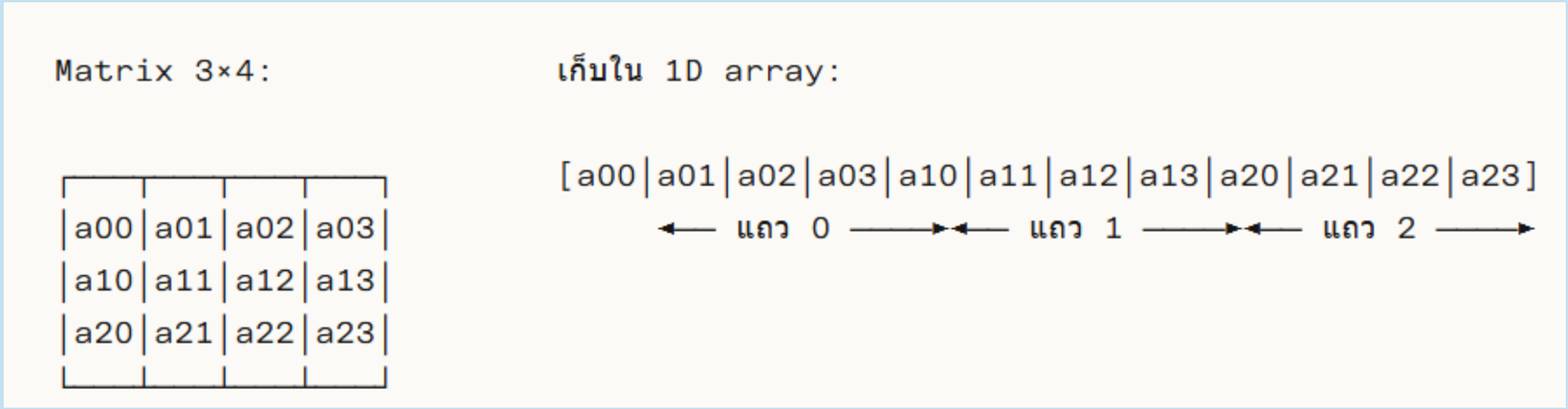
- “amortized” ไม่เหมือนกับ “average case” (กรณีทั่วไป)
- Average case หมายถึงการวิเคราะห์ว่า “โดยทั่วไปแล้ว input แบบไหนที่พบได้บ่อยที่สุด”
  - เช่น การค้นหาใน list ที่ไม่ได้เรียงลำดับ ในกรณีทั่วไปอาจจะเจอ target อยู่กลางๆ list ไม่ใช่ต้นหรือท้ายสุด
- Amortized หมายถึงการวิเคราะห์ ต้นทุนสะสมของการทำ operation เดียวกันซ้ำๆ หลายครั้งติดต่อกัน แล้วหารเฉลี่ย
  - ไม่ได้สนใจว่า input แต่ละครั้งจะเป็นแบบไหน แต่สนใจว่า ลำดับการทำงานทั้งหมด (เช่น การ append 1,000 ครั้งติดกัน) ให้ผลลัพธ์เฉลี่ยเท่าไร
- การ append ที่ trigger resize นั้น ไม่ใช่ worst case ที่เกิดแบบสุ่ม แต่เป็นเหตุการณ์ที่คาดเดาได้ล่วงหน้าและเกิดขึ้นเป็นระยะๆ อย่างมีรูปแบบ (ทุกครั้งที่ capacity เต็มพอดี)



2D Arrays

แนวคิด Row-Major Order

- คำถามคือ หน่วยความจำของคอมพิวเตอร์เป็นเส้นตรงมิติเดียว (1D) เท่านั้น (เหมือนตู้ล็อกเกอร์ยาวเหยียดเรียงเดียว) แล้ว matrix ที่มีทั้งแถวและคอลัมน์ (2 มิติ) จะถูกเก็บลงในหน่วยความจำแบบ 1 มิติได้อย่างไร
- คำตอบคือใช้เทคนิคที่เรียกว่า row-major order
  - หลักการ คือ **นำแต่ละแถวของ** matrix **มาเรียงต่อกันเป็นเส้นตรงเส้นเดียว**
  - โดยเริ่มจากแถวที่ 0 ตามด้วยแถวที่ 1 แถวที่ 2 ไปเรื่อยๆ
  - ลองดู matrix ขนาด 3 แถว 4 คอลัมน์ (เขียนแทนด้วย 3×4)

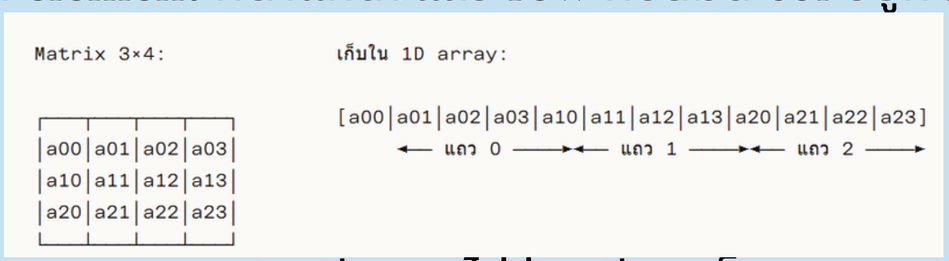


element ทั้งหมด 12 ตัว (3×4) ถูกนำมาเรียงต่อกันเป็นเส้นตรงเส้นเดียวใน 1D array

เริ่มจากแถวบนสุด (a00, a01, a02, a03) ต่อด้วยแถวถัดไป (a10, a11, a12, a13) และแถวสุดท้าย (a20, a21, a22, a23) ตามลำดับ

สูตรแปลงตำแหน่งจาก 2 มิติ เป็น 1 มิติ

- เมื่อข้อมูลถูกจัดเก็บแบบ row-major แล้ว การหาว่า element ที่ตำแหน่งแถว row คอลัมน์ col อยู่ที่ index ไหนใน 1D array ใช้สูตรนี้



  - flat_index = row × num_cols + col
    - โดย num_cols คือจำนวนคอลัมน์ทั้งหมดของ matrix (คงที่เสมอ ไม่ว่าจะกี่แถวก็ตาม)
    - **เหตุผล**: ถ้าต้องการไปถึงแถวที่ row ต้อง “ข้าม” แถวก่อนหน้าทั้งหมดไปก่อน ซึ่งแต่ละแถวมี num_cols ตัว ดังนั้นต้องข้ามไป row × num_cols ตำแหน่ง จากนั้นค่อยเลื่อนไปอีก col ตำแหน่งภายในแถวนั้นเพื่อถึง column ที่ต้องการพอดี
- เช่น ต้องการหา matrix[1][2] (แถวที่ 1 คอลัมน์ที่ 2) จาก matrix 3×4 ข้างต้น
  - flat_index = 1 × 4 + 2 = 6
  - จากภาพ ตำแหน่งที่ 6 ใน 1D array (นับจาก 0) คือ a12 ✓ ตรงกับที่ต้องการพอดี

ทำไม Row-Major Order ถึงสำคัญมาก

- มันทำให้ 2D array ยังคงมีความเร็วในการเข้าถึงเป็น O(1) เหมือน 1D array ทุกประการ
  - สูตร row × num_cols + col เป็นการคำนวณอย่างง่ายที่ใช้เวลาคงที่
  - ไม่ต่างจากสูตร base_address + i × element_size
  - แค่มีตัวแปรเพิ่มขึ้นมาอีกหนึ่งตัวเท่านั้น หลักการพื้นฐานยังคงเหมือนเดิมทุกประการ
- row-major order เป็นพื้นฐานสำคัญของ library ประมวลผลข้อมูลเชิงตัวเลขที่ใช้กันอย่างแพร่หลาย เช่น NumPy ซึ่งเป็น library หลักที่ใช้ในงาน Data Science และ Machine Learning
- ความเข้าใจเรื่อง row-major order จึงเป็นความรู้ที่ส่งผลโดยตรงต่อการเขียนโปรแกรมประมวลผลข้อมูลจริงในอนาคต

Complexity Summary

Search (การค้นหา) — ทำไมถึงเป็น O(n)

- Search แบบไม่ได้เรียงลำดับ (unsorted) คือ การค้นหาว่ามีค่าที่ต้องการอยู่ใน array หรือไม่ โดยไม่รู้ตำแหน่งล่วงหน้า
- ต่างจาก Access ที่รู้ index อยู่แล้วและคำนวณตำแหน่งได้ทันทีด้วยสูตร
- Search ไม่รู้ตำแหน่งล่วงหน้า จึงไม่มีทางเลือกอื่นนอกจาก ต้องสแกนตรวจสอบทีละ element ไปเรื่อยๆ ตั้งแต่ตัวแรกจนกว่าจะเจอ (หรือจนหมด array ถ้าไม่เจอเลย)
- ในกรณีที่แย่ที่สุด (ค่าที่หาอยู่ตำแหน่งสุดท้าย หรือไม่มีอยู่เลย) ต้องตรวจสอบครบทุก n ตัว ดังนั้น Search แบบ unsorted จึงเป็น O(n)
    - ยิ่ง array มีขนาดใหญ่ ยิ่งอาจต้องใช้เวลานานขึ้นตามสัดส่วน
    - ต่างจาก Access ที่คงที่เสมอไม่ว่า array จะใหญ่แค่ไหน

ตาราง Complexity Summary ฉบับสมบูรณ์

Operation	Time Complexity	คำอธิบาย
Access `arr[i]`	**O(1)**	คำนวณ address โดยตรงด้วยสูตร ไม่ต้องค้นหา
Search (unsorted)	**O(n)**	ต้องสแกนตรวจสอบทีละ element จนกว่าจะเจอ
Insert ที่ต้นหรือกลาง	**O(n)**	ต้องเลื่อน elements ทั้งหมดทางขวาของตำแหน่งนั้นไปทางขวาอีกที
Append (insert ท้าย)	**O(1) amortized**	ใช้ doubling strategy ตามที่เรียนในส่วนที่ 3-4
Delete ที่ต้นหรือกลาง	**O(n)**	ต้องเลื่อน elements ทั้งหมดทางขวาของตำแหน่งนั้นมาทางซ้าย
Delete ท้าย	**O(1)**	แค่ลด size ลงหนึ่งค่า ไม่ต้องเลื่อนอะไรเลย

Insert/Delete กลาง DynamicArray

ทำไมต้อง "เลื่อน" ข้อมูล

- array ต้องเก็บข้อมูลแบบต่อเนื่องกันในหน่วยความจำเสมอ (contiguous memory)
  - ไม่มีช่องว่างคั่นระหว่าง element เลยแม้แต่ตำแหน่งเดียว นี่คือสิ่งที่ทำให้ access เป็น O(1) ได้
  - แต่กฎเดียวกันนี้เองที่สร้างปัญหาเมื่อต้องแทรกหรือลบข้อมูลตรงกลาง
- ลองนึกภาพแถวเก้าอี้ในโรงหนังที่นั่งติดกันสนิท
  - ถ้ามีคนอยากจะ "แทรก" เข้าไปนั่งตรงกลางแถว
  - ทุกคนที่นั่งอยู่ทางขวาของตำแหน่งนั้นต้องขยับตัวไปทางขวาทีละคนก่อน ถึงจะมีที่ว่างให้คนใหม่นั่งได้
  - นี่คือสิ่งที่เกิดขึ้นกับ insert_at() ทุกประการ

Walkthrough insert_at() ทีละขั้น

- เริ่มจาก array [10, 20, 30, 40] (size=4, capacity=4) ต้องการแทรก 99 ที่ index 1
- เช็คว่ามีที่ว่างไหม เนื่องจาก size (4) เท่ากับ capacity (4) พอดี ต้อง resize ก่อน (capacity กลายเป็น 8) ได้ array [10, 20, 30, 40, _, _, _, _] ที่มีที่ว่างแล้ว
- จากนั้นเริ่มเลื่อนข้อมูล ทีละขั้นตอนย่อย โดยไล่จากตำแหน่งขวาสุดของข้อมูลจริง (size - 1 = 3) เข้าหาตำแหน่งที่จะแทรก (index = 1)
- ขั้นย่อยที่ 1 — คัดลอกค่าที่ index 3 (40) ไปไว้ที่ index 4
  - ก่อน: [10, 20, 30, 40, _, _, _, _]
  - หลัง: [10, 20, 30, 40, 40, _, _, _] ← ตำแหน่ง 4 ได้ค่า 40 มา (ยังมี 40 ซ้ำอยู่ที่ 3 ด้วย)
- ขั้นย่อยที่ 2 — คัดลอกค่าที่ index 2 (30) ไปไว้ที่ index 3
  - ก่อน: [10, 20, 30, 40, 40, _, _, _]
  - หลัง: [10, 20, 30, 30, 40, _, _, _] ← ตำแหน่ง 3 ถูกเขียนทับด้วย 30

Walkthrough insert_at() ทีละขั้น

- ขั้นย่อยที่ 3 — คัดลอกค่าที่ index 1 (20) ไปไว้ที่ index 2
  - ก่อน: [10, 20, 30, 30, 40, _, _, _]
  - หลัง: [10, 20, 20, 30, 40, _, _, _] ← ตำแหน่ง 2 ถูกเขียนทับด้วย 20
- หยุดที่นี่ เพราะ loop เขียนไว้ว่า range(size, index, -1) คือ range(4, 1, -1) ซึ่งได้ i = 4, 3, 2 เท่านั้น (หยุดก่อนถึง index 1) ตำแหน่ง index 1 ยังคงเป็นค่าเดิม (20) รออยู่
- ขั้นสุดท้าย — ใส่ค่าใหม่ 99 ลงในตำแหน่ง index 1 ที่ตอนนี้ "ว่าง" (มีข้อมูลเก่าอยู่ แต่ถูกคัดลอกไปที่อื่นแล้ว)
  - [10, 99, 20, 30, 40, _, _, _] → size = 5

สรุปภาพรวมการเลื่อนแบบ Right-to-Left

```
ตำแหน่ง:      0     1     2     3     4
เริ่มต้น:     10    20    30    40    _
                    ↑
              จะแทรกตรงนี้

ขั้นที่ 1:   10    20    30    40 → 40    (index 3 ไป 4)
ขั้นที่ 2:   10    20    30 → 30    40    (index 2 ไป 3)
ขั้นที่ 3:   10    20 → 20    30    40    (index 1 ไป 2)
ใส่ค่าใหม่:  10    99    20    30    40    (99 ลงที่ index 1)
```

- สังเกตทิศทางลูกศร — จากซ้ายไปขวา และเริ่มจากตำแหน่งขวาสุดก่อนเสมอ

ทำไมถ้าเลื่อนจากซ้ายไปขวาแทนถึงพัง

- ถ้าเขียน loop สลับทิศทาง (ไล่จาก index ไปทาง size แทน)

```
def insert_at_wrong(self, index, value):
    # ผิด — เลื่อนจากซ้ายไปขวา
    for i in range(index, self._size):
        self._data[i + 1] = self._data[i]
    self._data[index] = value
    self._size += 1
```

ทดสอบด้วยข้อมูลเดิม [10, 20, 30, 40] แล้ว insert_at_wrong(1, 99)
ผลลัพธ์จริงที่ได้

- ก่อน insert: [10, 20, 30, 40]
- หลัง insert_at_wrong(1, 99): [10, 99, 20, 20, 20]
- ควรได้ [10, 99, 20, 30, 40] แต่กลับได้ [10, 99, 20, 20, 20] — ข้อมูล 30 และ 40 หายไปเลย กลายเป็น 20 ซ้ำกันสามตัว

นี่คือเหตุผลที่ ต้องเริ่มเลื่อนจากขวาสุดก่อนเสมอ เพื่อไม่ให้ค่าที่ยังไม่ทันย้ายถูกเขียนทับไปก่อน

Walkthrough remove_at() ทีละขั้น

- เริ่มจาก array [10, 99, 20, 30, 40] (size=5) ต้องการลบค่าที่ index 1 (คือ 99)
- เก็บค่าที่จะลบไว้ก่อน — removed_value = 99
- จากนั้นเลื่อนข้อมูล ทีละขั้นตอนย่อย โดยไล่จากตำแหน่งที่ลบ (index = 1) ไปทางขวาจนถึงก่อนตัวสุดท้าย (size - 2 = 3)
- ขั้นย่อยที่ 1 — คัดลอกค่าที่ index 2 (20) มาไว้ที่ index 1
  - ก่อน: [10, 99, 20, 30, 40]
  - หลัง: [10, 20, 20, 30, 40] ← index 1 ถูกเขียนทับด้วย 20
- ขั้นย่อยที่ 2 — คัดลอกค่าที่ index 3 (30) มาไว้ที่ index 2
  - ก่อน: [10, 20, 20, 30, 40]
  - หลัง: [10, 20, 30, 30, 40] ← index 2 ถูกเขียนทับด้วย 30
- ขั้นย่อยที่ 3 — คัดลอกค่าที่ index 4 (40) มาไว้ที่ index 3
  - ก่อน: [10, 20, 30, 30, 40]
  - หลัง: [10, 20, 30, 40, 40] ← index 3 ถูกเขียนทับด้วย 40
- เคลียร์ตำแหน่งสุดท้าย และลด size ลง — [10, 20, 30, 40] (size=4) พร้อมคืนค่า 99 กลับไป

สรุปภาพรวมการเลื่อนแบบ Left-to-Right

```
ตำแหน่ง:      0     1     2     3     4
เริ่มต้น:     10    99    20    30    40
                    ↑
                 จะลบตรงนี้

ขั้นที่ 1:   10    20 ← 20    30    40    (index 2 มาแทน 1)
ขั้นที่ 2:   10    20    30 ← 30    40    (index 3 มาแทน 2)
ขั้นที่ 3:   10    20    30    40 ← 40    (index 4 มาแทน 3)
ผลลัพธ์:     10    20    30    40    (เคลียร์ตำแหน่งสุดท้าย, size ลด 1)
```

- สังเกตว่าทิศทางลูกศรตอนนี้กลับด้านจาก insert_at()
  - insert_at() เลื่อนขวาไปซ้าย (เริ่มจากท้าย)
  - ส่วน remove_at() เลื่อนซ้ายไปขวา (เริ่มจากตำแหน่งที่ลบ)
- หลักการจำง่ายๆ คือ insert ต้อง “เปิดช่องว่าง” จึงเริ่มจากท้ายไล่เข้ามา ส่วน remove ต้อง “ปิดช่องว่าง” จึงเริ่มจากจุดที่ลบไล่ออกไป

pop(): ทำไมเป็น O(1) โดยอัตโนมัติ

- pop() เขียนเพียงบรรทัดเดียวคือ return self.remove_at(self._size - 1)
- เมื่อ index เท่ากับ size - 1 (ตำแหน่งสุดท้ายพอดี) loop เลื่อนข้อมูลจะกลายเป็น range(size-1, size-1) ซึ่งเป็นช่วงว่างเปล่า
  - ไม่มีการวนแม้แต่รอบเดียว ตำแหน่งไม่มี element ใดต้องเลื่อนเลยเพราะไม่มีใครอยู่ทางขวาของตำแหน่งสุดท้ายอยู่แล้ว
- นี่คือเหตุผลที่ยืนยันทฤษฎีเรื่อง “delete ท้าย array เป็น O(1)”

Stack

Stack คืออะไร: หลักการ LIFO

- Stack คือโครงสร้างข้อมูลที่ทำงานตามหลัก LIFO (Last In, First Out)
  - สิ่งที่เข้าไปหลังสุด จะถูกนำออกมาก่อน เสมอ
- ตรงข้ามกับ Queue (คิว) ที่ทำงานแบบ FIFO (First In, First Out) ที่คุ้นเคยจากการต่อคิว
- Stack ไม่มีการ “แซมกลาง” หรือ “หยิบจากล่างสุด” ได้เลย
- มีเพียง จุดเดียว ที่ทั้งเพิ่มและนำออกได้ เรียกว่า “บนสุดของ stack” (top)

ตัวอย่าง Stack ในชีวิตจริง

- กองจานซ้อนกัน
  - เวลาล้างจานเสร็จแล้ววางซ้อนกันเป็นกอง จานใบล่าสุดที่วางจะอยู่บนสุด และเวลาหยิบมาใช้ก็ต้องหยิบจากบนสุดก่อนเสมอ
  - ไม่มีทางหยิบจานใบล่างสุดออกมาได้โดยไม่ยกจานที่ซ้อนอยู่ด้านบนออกก่อน
- ปุ่ม Undo ในโปรแกรมแก้ไขเอกสาร
  - ทุกครั้งที่พิมพ์หรือแก้ไข การกระทำนั้นจะถูก “push” เข้า stack
  - เมื่อกด Undo ระบบจะ “pop” การกระทำล่าสุดออกมายกเลิกก่อนเสมอ
  - ไม่สามารถข้ามไปยกเลิกการกระทำที่ทำไว้เมื่อ 10 ครั้งก่อนได้โดยตรง
- ปุ่ม Back ของเบราว์เซอร์
  - ทุกหน้าเว็บที่เข้าชมจะถูก push เข้า stack
  - เมื่อกด Back จะ pop หน้าล่าสุดออกไปแล้วกลับไปหน้าก่อนหน้าเสมอ

Operations หลัก: push และ pop

- push(value) เพิ่มค่าใหม่ไว้ บนสุด ของ stack เสมอ

```
เริ่มต้น:          []
push(10):          [10]
push(20):          [10, 20]
push(30):          [10, 20, 30]
                              ↑
                           บนสุด (top)
```

pop() นำค่า บนสุด ออกจาก stack แล้วคืนค่านั้นกลับไป

```
ก่อน pop:   [10, 20, 30]
pop() คืนค่า 30
หลัง pop:   [10, 20]
                  ↑
               บนสุดใหม่ (top)
```

สังเกตว่า pop() เอาออกได้แค่ 30 เท่านั้น (ตัวล่าสุดที่ push เข้าไป) ไม่สามารถ pop 10 ออกมาได้เลยตราบใดที่ยังมี 20 และ 30 ค้างอยู่ด้านบน – นี่คือหัวใจของ LIFO

Operations เสริม: peek และ is_empty

- peek() แอบดู ค่าบนสุดโดยไม่เอาออก ต่างจาก pop() ตรงที่ stack ยังคงมีขนาดเท่าเดิมหลังเรียก

```
ก่อน peek: [10, 20, 30]
peek() คืนค่า 30 (แค่ดู ไม่เอาออก)
หลัง peek: [10, 20, 30]    ← ขนาดเท่าเดิม ไม่เปลี่ยนแปลง
```

- is_empty() ตรวจสอบว่า stack ว่างเปล่าหรือไม่ สำคัญมากเพราะต้องเช็คก่อนเรียก pop()/peek() เสมอ (เหมือนที่เรียนมาแล้วในโจทย์ Lab ว่าต้อง raise error ถ้า stack ว่าง)

“บนสุดของ Stack” ตรงกับ “ท้ายสุดของ Array”

- นี่คือจุดเชื่อมโยงสำคัญที่สุดระหว่าง Stack กับ DynamicArray ที่สร้างไว้แล้ว
- ลองนึกดูว่า append() ของ DynamicArray เพิ่มค่าใหม่ที่ตำแหน่งท้ายสุดเสมอ (_data[_size]) และ pop() ของ DynamicArray ก็เอาตำแหน่งท้ายสุดออกเสมอเช่นกัน (_data[_size - 1])
  - พฤติกรรมนี้ ตรงกับ LIFO ของ Stack แบบเป๊ะๆ ไม่ต้องดัดแปลงอะไรเลย
- DynamicArray.append(value) ≡ Stack.push(value)
- DynamicArray.pop() ≡ Stack.pop()
- นี่คือเหตุผลที่ Stack.push() และ Stack.pop() ที่เขียนในโจทย์ Lab สามารถ เรียก DynamicArray.append()/DynamicArray.pop() ตรงๆ ได้เลย โดยไม่ต้องคำนวณตำแหน่งอะไรเพิ่มเติมทั้งสิ้น

เปรียบเทียบ Stack กับ DynamicArray ปกติ

ความสามารถ	DynamicArray	Stack
เข้าถึงตำแหน่งใดก็ได้ (`arr[5]`)	✅ ทำได้	❌ ทำไม่ได้เลย
แทรก/ลบตรงกลาง (`insert_at`, `remove_at`)	✅ ทำได้	❌ ทำไม่ได้เลย
เพิ่ม/ลบเฉพาะปลายด้านเดียว	✅ ทำได้ (แต่ไม่บังคับ)	✅ ทำได้ (และบังคับเท่านั้น)
ดูค่าบนสุดโดยไม่เอาออก	ไม่มี method นี้โดยตรง	✅ `peek()`

ทำไมต้อง “ห่อ” (Wrap) ด้วย class ใหม่แทนที่จะใช้ DynamicArray ตรงๆ

- ในเมื่อ Stack แค่เรียก append()/pop() ของ DynamicArray ตรงๆ
  - ทำไมไม่ใช้ DynamicArray เป็น Stack ไปเลย ไม่ต้องสร้าง class ใหม่?
- ถ้าใช้ DynamicArray ตรงๆ โค้ดภายนอกจะยังคงเรียก arr.insert_at(2, x) หรือ arr[5] ได้อย่างอิสระ ซึ่งขัดกับเจตนาของ Stack ที่ต้องการ “บังคับ” ให้ทำงานผ่านปลายด้านเดียวเท่านั้น
- การสร้าง Stack เป็น class แยกต่างหาก (ที่มี DynamicArray เป็นส่วนประกอบแบบ Composition) คือ การจำกัดความสามารถโดยเจตนา
  - เปิดเผยเฉพาะ push, pop, peek, is_empty เท่านั้น
  - ไม่เปิด insert_at, __getitem__, หรือ method อื่นของ DynamicArray ออกมาให้ใช้เลย
- นี่คือหลักการที่เรียกว่า “จำกัด interface ให้ตรงกับพฤติกรรมที่ต้องการ”
  - ยิ่ง interface แคบและชัดเจนเท่าไร ยิ่งป้องกันการใช้งานผิดวัตถุประสงค์ได้ดีเท่านั้น

การวน: Composition

Composition

- คือ ความสัมพันธ์ที่ class หนึ่ง "ประกอบด้วย" อีก class หนึ่งอย่างแนบแน่น
- ส่วนประกอบถูกสร้างขึ้นภายในเจ้าของโดยตรง ไม่ได้รับมาจากภายนอก และไม่มีทางดำรงอยู่ได้เลยหากไม่มีเจ้าของ
- เกณฑ์ตัดสินใจ 2 ข้อ
  - ส่วนประกอบถูกสร้างจากภายนอกแล้วส่งเข้ามา หรือสร้างขึ้นภายในเจ้าของเอง?
  - ถ้าเจ้าของถูกทำลาย ส่วนประกอบนั้นยังมีความหมายอยู่หรือไม่?
- สัญญาณที่สังเกตได้จาก syntax — Composition:
  - constructor/method รับแค่ ข้อมูลดิบ (string, number) แล้วสร้าง object เองภายใน
  - เช่น House.add_room(name, area) ที่สร้าง Room ขึ้นภายในเอง ไม่รับ Room object สำเร็จรูปเข้ามา

เชื่อมสู่โจทย์ Lab: Stack

- คือ ความสัมพันธ์ที่ class หนึ่ง "ประกอบด้วย" อีก class หนึ่งอย่างแนบแน่น
- ส่วนประกอบถูกสร้างขึ้นภายในเจ้าของโดยตรง ไม่ได้รับมาจากภายนอก และไม่มีทางดำรงอยู่ได้เลยหากไม่มีเจ้าของ
- เกณฑ์ตัดสินใจ 2 ข้อ
  - ส่วนประกอบถูกสร้างจากภายนอกแล้วส่งเข้ามา หรือสร้างขึ้นภายในเจ้าของเอง?
  - ถ้าเจ้าของถูกทำลาย ส่วนประกอบนั้นยังมีความหมายอยู่หรือไม่?
- สัญญาณที่สังเกตได้จาก syntax — Composition:
  - constructor/method รับแค่ ข้อมูลดิบ (string, number) แล้วสร้าง object เองภายใน
  - เช่น House.add_room(name, area) ที่สร้าง Room ขึ้นภายในเอง ไม่รับ Room object สำเร็จรูปเข้ามา